Cross-Schema Composition for Attestation-Native Chains

Typed Attestation References with Slashing-Aware Proof Propagation

Ligate Labs Research, Working Paper v0.2

Date: 2026-05-22

Contact: hello@ligate.io

Abstract

Ethereum smart contracts can reference arbitrary chain state by hash. The reference is well-typed if and only if the consumer contract enforces the type, which it does as application logic. There is no chain-level guarantee that a Solidity contract is consuming the right kind of input. Slashing propagation through references is also entirely application-level: contract A may invalidate state that contract B reads, and contract B has no native machinery to learn this. The result, across the EVM ecosystem, is a steady stream of integration bugs and a high cost of audit for any contract that composes with another contract's state.

This paper specifies **chain-enforced typing** for attestation references and **slashing-aware proof propagation** through the schema dependency graph as a runtime primitive on Ligate Chain. A schema declares its input dependencies as part of its registration: which schemas it may reference, at which version constraints, with which predicate over the input payloads. The runtime rejects, at mempool admission, any attestation that references an input of the wrong type, the wrong version, or invalid status. When a referenced attestation transitions to REVOKED, SLASHED, or DEPENDENT-INVALID, the runtime propagates the transition through the dependency graph according to each schema's declared cascade rule (strict or lazy), and the §5.5 termination theorem guarantees the cascade completes in bounded time.

Three contributions. First, we specify the protocol-level mechanism (§3 and §4): the schema-as-typed-graph model, the input-type-set with semver-constrained edges, the deterministic type predicate, and the runtime type check at mempool admission. Second, we prove the **slashing-cascade termination theorem (§5.5)**: under acyclic dependency graphs with depth d, the cascade triggered by one slash event terminates in $O(d)$ deterministic steps with bounded per-step compute, with explicit per-edge deduplication that prevents re-cascading through the same edge twice. Third, we analyze three attack families (type confusion, slash amplification, cycle-induced DoS) and bound the damage each can inflict (§8).

The mechanism is positioned as a **v2 protocol feature**. v1 of Ligate Chain ships with single-schema attestations only; cross-schema composition lands when 2-3 design partners have asked for it with concrete use-case descriptions matching the §6.2 template. This paper exists so the design space is captured before then, not as a roadmap commitment to ship. §6 is explicit about the use-case-validation gate: without validated design-partner demand, the mechanism remains specification-only.

---

1. Introduction

1.1 The Composition Thesis

Attestations are claims about things. The simplest kind is a claim about an external fact: "I observed event X," "the prompt for this AI output was P," "this content was authored by Y." But many natural claims are claims about *other claims*. "This AI output was produced by model M from prompt P at time T" is a claim that references three other claims: the model attestation, the prompt attestation, and the timestamp attestation. "This DAO membership inherits from this identity proof" is a claim about a claim. "The audit of this RWA reserve was conducted by this licensed auditor" is a claim about a claim. The category is large enough that any production attestation chain will encounter it.

When attestations reference other attestations, the chain has a choice. It can treat the reference as an opaque pointer to chain state and let application code enforce the type contract (this is Ethereum's model: smart contracts read each other's storage by hash and trust the hash to point at the right kind of thing). Or it can enforce the type contract at the chain layer, rejecting attestations that reference inputs of the wrong type or invalid status before they reach consensus.

The thesis of this paper: for an attestation-native chain, **chain-enforced typing is the right place to put the type contract**. Reasons in §1.3.

1.2 Why Now (or Why Not)

Honest framing first: cross-schema composition is **v2 protocol territory**. Ligate Chain v0 ships with single-schema attestations only. Themisra (proof of prompt), Mneme (wallet receipts), Iris (agent attestations), Atlas (verifier results): each of the four flagship products at v1 operates on its own schema with no need to reference attestations of other schemas. The single-schema primitive is sufficient for the first wave of products, and v1 engineering cost is finite.

What changes in v2 is the gravitational pull of composition. Once the chain has 50+ registered schemas, the natural next layer is composition: workflows that consume attestations of one schema and produce attestations of another. Themisra wants to produce attribution attestations referencing prompt attestations. Iris wants to produce agent-action attestations referencing the Themisra attribution they were responding to. Compliance partners want to produce audit attestations referencing the financial attestations they audited. The pattern is universal once the schema count crosses a threshold.

This paper exists so the design is captured *before* the engineering cycle starts. v0.2 lands the mechanism, the theorem, the security analysis, and the comparison table. §6 is the gate: actual engineering work begins only when 2-3 design partners have submitted concrete use-case descriptions matching the §6.2 template. Until then, this paper is specification-only.

1.3 The Type-Confusion Problem

A worked example clarifies why chain-enforced typing matters.

Consider a consumer schema `themisra.proof-of-attribution/v1` that produces attestations of the form: "this AI output was produced from this prompt by this model." The schema declares an input-type set: `{themisra.proof-of-prompt/v1, themisra.model-registration/v1}`. The consumer's predicate evaluates: "the input prompt attestation's `prompt_id` field matches my own `prompt_id`; the input model attestation's `model_hash` matches my own `model_hash`; my own `output_hash` is provided in the payload."

Without chain-enforced typing, the consumer takes any 32-byte attestation-id and trusts it points at the right kind of thing. An adversary submits an attestation-id that points to, say, a Mneme transfer receipt. The bytes happen to deserialize as something the consumer's predicate can parse (Mneme receipts have a `from_addr` field, the consumer reads it expecting a `prompt_id`, the field happens to be 32 bytes, parsing succeeds). The consumer's predicate evaluates against the wrong inputs and may produce `true` for an attribution claim that is structurally meaningless. The chain accepts the attestation. Downstream applications treat the bogus attribution as canonical.

With chain-enforced typing, step 3 of the §4.4 runtime check rejects the submission at admission time: "input type mismatch." The Mneme transfer receipt is not of schema

`themisra.proof-of-prompt/v1`, the chain knows this, the attestation never lands. The adversary cannot construct a successful attack without forging an attestation under the correct schema (which is harder, and is bounded by PoUA's slashing).

The same pattern recurs across application domains. Identity composition: a DAO-membership attestation that references the wrong kind of identity proof is a privilege-escalation. Financial conservation: a transfer attestation that references the wrong kind of balance proof is a counterfeiting risk. Audit trails: an audit attestation that references the wrong kind of audited document is a fraud surface. Without chain-enforced typing, every consumer re-implements its own type check, the checks compound errors across the system, and audit costs scale linearly in composition depth.

1.4 The Central Question

> **What is the minimum typing and slashing-propagation primitive that makes cross-schema composition safe to use in production, while remaining cheap enough to ship in a runtime, and gated cleanly enough that workloads which do not need it pay no overhead?**

This paper answers: schemas declare input-type sets at registration with semver-constrained edges; the runtime checks types at mempool admission; slashing cascades through the dependency graph with bounded depth and deterministic order; cycles are statically rejected by default.

1.5 Approach in Brief

Schemas register with a declared input-type set: which schemas they may reference, at which version constraints, with which boolean predicate over the input payloads (§4.1, §4.2, §4.3). The runtime checks each submitted attestation at mempool admission (§4.4): schema lookup, reference resolution, input-type check, validity check, predicate evaluation, plus the existing PoUA checks. Attestations that fail any check are rejected before block inclusion.

When an attestation transitions to REVOKED, SLASHED, or DEPENDENT-INVALID, the runtime cascades the state change through the dependency graph (§5). Cascade rules are configurable per-schema (strict or lazy). The §5.5 termination theorem guarantees the cascade completes in $O(d)$ deterministic steps where d is the dependency graph's depth. Cycles are statically rejected at registration (§5.6); a v1+ dynamic mode allows cycles with a bounded `max_cascade_depth` and a `cycle_break_rule` parameter.

The mechanism composes orthogonally with PoUA's reputation accounting (reputation accrues to the attestor of the consumer schema, independent of the input schemas), per-schema fee markets (each schema in the dependency chain pays its own base fee at its own utilization curve), and native delegation (a hot key with a grant scoped to schema σ can submit attestations of σ that reference other schemas, but only schemas explicitly enumerated in the grant's scope predicate).

1.6 Contributions

The paper makes four contributions.

A **mechanism specification** in §3 and §4: the schemas-as-typed-graphs model, the input-type set with semver-constrained edges, the deterministic type predicate with bounded compute, the runtime type check at mempool admission, and the schema versioning rules (strict / subtyping / per-edge override).

A **slashing-cascade termination theorem (§5.5)**: under acyclic dependency graphs with maximum depth $d_{\max}$, the cascade BFS terminates in $O(d)$ deterministic steps with explicit per-dependent deduplication. Cycles are statically rejected by default; an opt-in dynamic mode is specified for v1+ with `cycle_break_rule` and `max_cascade_depth` parameters.

A **security analysis (§8)** of three attack families: type-confusion attacks (rejected at admission), slash-amplification attacks (gas charged to slash root, not dependents), cycle-induced DoS (static rejection plus depth bound). Each family is bounded with explicit cost analysis.

A **use-case-validation gate (§6)**: explicit framing that this paper is specification-only until 2-3 design partners submit concrete use-case descriptions matching the §6.2 template. The mechanism is interesting and probably correct, but the engineering cost is justified only by validated demand. v0.2 captures the design space, not a roadmap commitment.

1.6.1 Status of Claims Following the PoUA v0.7 + native-delegation v0.2 + per-schema-fees v0.2 discipline:

Proven (formal mathematical argument under standard assumptions):

- §5.5 slashing-cascade termination theorem: under acyclic graphs, BFS terminates in $O(d)$ steps with deterministic per-dependent deduplication.

Bounded under stated assumptions:

- §4.4 type check correctness assumes the schema's `predicate` field is deterministic (the runtime verifies this via dry-run on canonical inputs at registration time; non-deterministic predicates are rejected). Under-the-assumption guarantee.
- §8.3 slash-amplification defense assumes the gas-accounting model from §5.7 (each slash root pays its own cascade cost). If a future architecture separates proposer from builder, the accounting needs revision.
- §5.6 cycle handling: static-mode correctness is unconditional; dynamic-mode termination assumes the runtime enforces `max_cascade_depth` and `cycle_break_rule` correctly. Trust boundary explicit.

Empirical or heuristic, requiring real-world demand:

- §6 use-case validation: not a chain claim; a process claim. The paper claims the mechanism is correct under the §5.5 theorem; whether the mechanism is *useful* depends on whether real workloads need it. v0.2 of the paper does not claim this; it sets the gate at "2-3 design partners with concrete use cases."

1.7 Scope and Non-Goals

In scope:

- Typed attestation references for cross-schema composition on Ligate Chain (single chain)
- Schema declaration syntax and registration validation
- Runtime type check at mempool admission
- Slashing-cascade rules and the termination theorem
- Cycle handling (static + opt-in dynamic)
- Security analysis under type-confusion, slash-amplification, cycle-DoS

Explicitly out of scope:

- **Cross-chain references.** An attestation on Ligate Chain referencing one on Ethereum (or vice versa) is the cross-chain composition problem; a separate paper covers IBC-mediated attestation references.
- **Zero-knowledge predicate types.** Predicate types that hide the input payloads while proving validity (Zk-SNARK / Zk-STARK) are research-grade and deferred to a follow-up paper.
- **Fully-dependent type systems.** Coq / Agda / Idris / Lean offer arbitrarily expressive types but require proof-search at type-check time. Inappropriate for a chain runtime; included in §2.4 only as design-space context.
- **Privacy-preserving references.** A reference reveals the dependency edge in public chain state. Use cases requiring private references (e.g., the consumer schema does not want to disclose which input it consumed) need additional cryptography; §9.4 names this as future work.
- **Multi-resource per-attestation pricing.** The per-schema fees paper (companion) handles per-workload pricing; the within-schema multi-resource axis is deferred to a separate paper.

1.8 Document Structure

Section 1.6.1 separates the paper's claims into proven, bounded-under-stated-assumptions, and empirical-or-heuristic; readers in a hurry may want to start there. Section 2 surveys smart-contract reference patterns, EAS schema graph, capability-secure languages, dependent types in programming languages, and recursive invalidation patterns from distributed systems. Section 3 fixes the system model: schemas as typed graphs, dependency graph, attestation as witness, validity states. Section 4 specifies the type system: schema declaration, input-type set, type predicate, runtime check, versioning semantics. Section 5 specifies slashing propagation: strict and lazy cascade, configurability, the §5.5 termination theorem, cycle handling, concurrent invalidation races. Section 6 documents the use-case validation gate. Section 7 positions cross-schema composition against prior systems. Section 8 analyzes three attack families. Section 9 lists limitations and future work; Section 10 concludes.

---

2. Background and Related Work

Cross-system references with type and validity propagation is a recurring problem across distributed systems, smart contracts, programming languages, and database theory. This section surveys five families of related work, each illuminating a different facet of the design space.

2.1 Smart-Contract Reference Patterns

Ethereum smart contracts reference each other's state and tokens via opaque hashes plus interface conventions. ERC-721 token references identify NFTs by `(contract_addr, token_id)`; the consumer contract calls `ownerOf(token_id)` and trusts the call's return value. ERC-1155 generalizes to multi-token references; ERC-4907 layers rental semantics on top. In each case, the chain checks that the call targets the declared contract address (a structural check) but does not check that the contract at that address implements the right interface (a type check). A consumer that calls `ownerOf` on a contract that does not implement ERC-721 gets whatever the contract returns at the same function selector, or a revert, depending on the implementation.

This is application-level typing. It works because (i) Solidity developers are conditioned to check interface support before calling, and (ii) tools like OpenZeppelin's safe-call patterns codify the check. But the chain itself has no opinion: a contract pointing at any other contract is valid, and the consumer is responsible for catching mismatches. Compositional bugs across ERC-721 + ERC-1155 + ERC-4907 are routine in audit reports.

What ERC patterns offer that this paper does not. Arbitrary contract logic at the consumer side. A consumer can implement arbitrarily sophisticated interface detection or fallback patterns.

What this paper offers that ERC patterns do not. Chain-level type enforcement at attestation-time. The consumer doesn't need to implement interface detection; the chain rejects mismatches at admission.

2.2 EAS Schema Graph

Ethereum Attestation Service (EAS) is the closest existing analog to what this paper specifies. EAS supports cross-schema references via the `refUID` field in each attestation: a 32-byte identifier pointing to a referenced attestation. Schemas are first-class on-chain entities; the `refUID` points to an attestation of some schema.

EAS's type-check is **application-level**, not chain-level. The schema declaration specifies the expected payload structure, but it does not enforce a typed input set. A consumer schema that wants to reference a specific type of input attestation has to validate the input's schema-ID in its own attestation logic, off-chain or in a downstream contract.

EAS's slashing-cascade is also **application-level**: there is no native machinery for "if the input is revoked, the consumer becomes invalid." Revocation is a per-attestation flag readable by anyone; downstream contracts that care must check it themselves.

What EAS offers that this paper does not. Production deployment on Ethereum mainnet since 2022. Stable interfaces, well-known to the Ethereum developer community.

What this paper offers that EAS does not. Chain-enforced typing (the `refUID` must be of the declared input schema) and chain-enforced slashing cascade (dependents transition automatically per the registered cascade rule). EAS users implementing these by hand at the application layer pay engineering cost and audit cost; this paper moves both to the chain.

2.3 Capability-Secure Systems

Capability-based programming languages (E, Pony, Joe-E, Caja) provide compile-time enforcement of "object A can only invoke object B if it holds a capability for B." A capability is an unforgeable token granting specific authority. The compiler verifies, at type-check time, that every cross-object reference is mediated by a capability declared in the program's authority graph.

The intuition transfers to chain-level typing: a schema can only reference another schema if the dependency edge is declared at registration time. The "compile-time check" in capability languages corresponds to the "registration-time check" in this paper. Both prevent ad-hoc reference to other objects (or schemas) outside the explicitly-declared authority graph.

The conceptual ancestor argument. Capability-secure languages establish that authority-graph declaration plus compile-time enforcement is a clean way to bound cross-object reference. This paper applies the same architecture at chain-runtime granularity.

What capability languages offer that this paper does not. Full programming-language type systems (objects, methods, full structural typing). This paper specifies only the schema-to-schema edge, not richer in-schema typing.

What this paper offers that capability languages do not. Slashing-cascade semantics. Capability languages do not model the case where one object's authority is revoked and dependents must respond. This paper specifies the cascade explicitly (§5).

2.4 Dependent Types in Programming Languages

Dependent type systems (Coq, Agda, Idris, Lean) provide arbitrarily expressive types: a type can depend on a value. A vector of length n is a different type from a vector of length $n + 1$. Type-checking requires the compiler to *prove* that types align; for non-trivial programs, this is undecidable in general and requires programmer-provided proof terms.

The relevance to this paper: dependent types are the gold standard for type safety. If the chain were a Coq runtime, every type contract would be machine-checkable. The cost is impractical: proof terms are large, proof search is expensive (sometimes undecidable), and the developer experience requires expertise.

Design-space role. Dependent types define the upper bound of what's possible. This paper picks a much weaker point in the design space: structural types over schemas, deterministic bounded-compute predicates over payloads, no full proof-search. The trade-off is type expressiveness against runtime cost. Section 8 shows that the weaker design still prevents the practical attacks (type confusion, slash amplification, cycle DoS); full dependent typing would prevent strictly more, but at engineering cost the chain cannot afford.

2.5 Recursive Invalidation in Distributed Systems

Cascading invalidation is a recurring problem across distributed systems and database theory. Three relevant patterns:

Cascading deletes in relational databases. SQL's `ON DELETE CASCADE` propagates a row deletion to dependent rows through foreign-key edges. Termination is guaranteed by foreign-key acyclicity (most schemas enforce this); the cascade is depth-first by default with explicit transaction boundaries. This paper's strict cascade (§5.2) is close in spirit to `ON DELETE CASCADE`: both propagate deletion-style state changes through declared edges, both bound termination through structural constraints.

Cache invalidation in distributed key-value stores. A write to a cached value triggers invalidation messages to dependents (other caches holding the same key or computed derivatives). Termination and consistency are managed through versioning (each cache entry carries a version; invalidation messages reference the version) and timeouts. This paper's per-dependent deduplication (§5.7) borrows the versioning intuition: each dependent is invalidated at most once per cascade.

Transaction rollback in WAL (write-ahead-log) systems. Aborting a transaction propagates undo records through the WAL, undoing dependent operations in reverse-commit order. Strict serializability is preserved by ensuring the undo order is the inverse of the do order. This paper's BFS-based cascade is not WAL-style (no undo records), but the determinism-via-canonical-ordering principle (§5.7) is the same: a deterministic schedule prevents inconsistent observations across the system.

What this paper takes. Acyclicity-by-default for termination (from relational DBs), versioning-based deduplication (from cache invalidation), deterministic ordering (from WAL). The synthesis is the §5 mechanism: BFS cascade over declared edges with per-dependent deduplication and canonical-ID ordering.

---

3. System Model

This section formalises the typed-attestation-graph model: schemas as nodes, dependency edges as typed arrows, attestations as witnesses, and the validity state machine that supports slashing-aware cascades.

3.1 Schemas as Typed Graphs

A **schema** σ is a tuple

$$\sigma = (I_\sigma, O_\sigma, P_\sigma, \mathcal{A}_\sigma, V_\sigma)$$

where:

- I_σ is the **input-type set**: a finite list of (σ', v') pairs identifying schemas σ' at version constraints v' that this schema may reference (§4.2)
- O_σ is the **output payload type**: a structural schema (e.g., JSON Schema, Protobuf-like definition) for the attestation's payload
- P_σ is the **type predicate**: a deterministic function $P_\sigma : I_\sigma^* \times O_\sigma \to \{\text{true}, \text{false}\}$ that evaluates input attestations and payload to a validity boolean (§4.3)
- $\mathcal{A}_\sigma$ is the **attestor set** required to sign attestations of this schema (inherited from the protocol's existing schema primitive)
- V_σ is the **schema version**, a monotonic integer

A schema with $I_\sigma = \emptyset$ is a **leaf schema**: it does not reference other schemas. v0.7 PoUA chains support only leaf schemas; this paper specifies the extension to non-leaf schemas via the input-type-set I_σ.

3.2 Dependency Graph

The collection of registered schemas and their dependency edges forms a **directed graph**:

$$\mathcal{G} = (\Sigma, E)$$

where Σ is the set of registered schemas and $E \subseteq \Sigma \times \Sigma$ is the set of dependency edges. An edge $\sigma_a \to \sigma_b$ exists iff $(\sigma_b, v) \in I_{\sigma_a}$ for some version constraint v that admits σ_b's current version V_{σ_b}.

Acyclicity by default. Schema registration rejects edges that would close a cycle; cycle detection is computed at registration time via topological sort. This is the v0 default. §5.6 specifies an opt-in cyclic mode for advanced use cases.

Versioning expands the graph. When a schema upgrades from $V_{\sigma_b} = 1$ to $V_{\sigma_b} = 2$, the edges into σ_b are re-evaluated against the new version. Dependents that admit the new version

(via subtyping, §4.5) keep their edges; dependents that require strict version match must be re-registered.

Graph properties. At v0 scale (a few hundred schemas), $|\Sigma|$ and $|E|$ are small; cycle-check and topological-sort costs are negligible. At v1+ scale (thousands of schemas), incremental cycle detection on each new registration becomes the cost driver; v0.2 of this paper specifies the algorithm.

3.3 Attestation as Witness

An **attestation** a of schema σ is the tuple

$$a = (\sigma, K^{\text{signer}}, \text{payload}_a, \text{refs}_a, t_a, s_a)$$

where:

- σ is the schema-id (which fixes $I_\sigma, O_\sigma, P_\sigma, \mathcal{A}_\sigma, V_\sigma$)
- K^{signer} is the signing key (typically the threshold-signature aggregation of $\mathcal{A}_\sigma$)
- payload_a is the payload conforming to O_σ
- refs_a is the **reference list**: $(\text{att-id}_1, \text{att-id}_2, ...)$ indexing input attestations satisfying I_σ
- t_a is the inclusion height (block at which the attestation was finalized)
- s_a is the threshold signature

Witness semantics. An attestation a is a witness to "the predicate P_σ held over the referenced inputs and the stated payload at height t_a." The chain stores the attestation; light clients can verify the predicate held by re-evaluating P_σ at any future point.

Empty references. Leaf schemas have $\text{refs}_a = ()$ (empty tuple). The runtime trivially passes the input-type check for leaf schemas; this is how PoUA v0.7's existing schema primitive composes with this paper's typed extension without requiring chain-state migration.

Reference-by-id, not by-content. refs_a uses canonical attestation-ids, not payload digests. The id binds to a specific on-chain attestation rather than any attestation with a matching payload, eliminating ambiguity when payloads collide across schemas.

3.4 Validity States

Attestations live in a **validity state machine** with four states:

```
VALID -> REVOKED            (revocation by signer or attestor-set)
VALID -> SLASHED            (PoUA-side slashing event)
VALID -> DEPENDENT-INVALID  (cascade from a non-VALID input)
```

State transitions.

- **VALID** is the initial state on inclusion. The runtime type check (§4.4) must pass for the attestation to enter VALID.
- **VALID → REVOKED** fires when the schema's attestor set issues a revocation transaction. Used for retraction (e.g., "this attestation was issued in error").
- **VALID → SLASHED** fires when slashing rules per PoUA §4.5 detect misbehavior on the proposer or attestor set. Slashing applies the reputation penalty and transitions any included attestations to SLASHED.

- **VALID → DEPENDENT-INVALID** fires when at least one of the attestation's refs_a transitions out of VALID. The cascade rule (§5) determines whether the dependent attestation is auto-invalidated or whether application-layer logic decides.

Terminal states. REVOKED, SLASHED, and DEPENDENT-INVALID are terminal: no further transitions are possible. This simplifies the chain's storage model (each attestation has a single state-bit set).

Cascade rules. Whether DEPENDENT-INVALID fires automatically at each ref-transition is governed by the schema's declared cascade preference (§5.4). Strict cascade fires automatically; lazy cascade leaves the dependent VALID and exposes the ref-state via the read API for application-layer handling.

---

4. Type System

This section specifies how schemas declare types, how the runtime checks types, and how versioning interacts with the typed-graph model.

4.1 Schema Declaration

Schema registration submits a **schema-declaration object**:

```
SchemaDeclaration = {
  name:                string,   // canonical schema-id
  version:             int,      // monotonic version counter
  payload_schema:      Type,     // structural schema
  input_type_set:      List<(SchemaId, VersionConstraint)>,
  predicate:           Bytecode, // deterministic boolean function
  attestor_set_id:     Id,       // existing PoUA primitive
  cascade_rule:        enum {STRICT, LAZY},
  predicate_gas_limit: int,      // bounded compute cost (default 1000)
}
```

Field notes: `name` is the canonical schema-id (e.g. a `themisra.proof-of-prompt` schema at version `v1`); `payload_schema` is a structural type (JSON Schema or equivalent); `predicate` is a bytecode-encoded deterministic boolean function; `attestor_set_id` references the existing PoUA primitive.

Validation at registration time.

1. `name` is unique in Σ across all current versions.
2. `version` strictly exceeds all prior versions of `name`.
3. Each $(\sigma', v') \in$ `input_type_set` references an existing schema; cycle detection runs against $\mathcal{G}$ extended with the proposed edges (§3.2).
4. `predicate` is deterministic (verified by the runtime via dry-run on canonical inputs) and bounded by `predicate_gas_limit`.
5. `attestor_set_id` exists.
6. `cascade_rule` is one of the two enum values.

Failure of any check rejects the registration; partial registration is not allowed. Registration fees in `$AVOW` apply per PoUA §3.

4.2 Input Type Set

The **input-type set** I_σ enumerates which schemas an attestation of σ may reference. Each entry is a (schema-id, version-constraint) pair:

$$I_\sigma = \{(\sigma_1, v_1), (\sigma_2, v_2), \ldots, (\sigma_n, v_n)\}$$

where each v_i is one of:

- **Exact match** ($v_i = k$): only attestations of σ_i at version exactly k are admissible
- **Semver range** ($v_i = [k, k')$): versions $V \in [k, k')$ are admissible; supports forward-compatible consumers
- **Open upper** ($v_i = [k, \infty)$): all versions $\geq k$
- **Wildcard** ($v_i = *$): any registered version of σ_i. Discouraged; only use for cross-version-tolerant consumers (e.g., audit-log readers)

Why version constraints matter. A consumer schema declaring "I reference attestations of `themisra.proof-of-prompt/*`" accepts any future version of Themisra without re-registration. Convenient but risky: a v3 of Themisra with structurally-different payload could break the consumer's predicate. Exact match prevents this at the cost of forcing re-registration on every input upgrade. Semver ranges balance the two.

Empty input-type set. $I_\sigma = \emptyset$ marks σ as a leaf schema (§3.1). Attestations of leaf schemas have $\text{refs}_a = ()$ and the type check trivially passes the input checks.

4.3 Type Predicate

The **type predicate** P_σ is a deterministic boolean function:

$$P_\sigma : I_\sigma^* \times \text{Payload}_\sigma \to \{\text{true}, \text{false}\}$$

evaluated at attestation-time. Inputs to the predicate:

- Each input attestation's schema, version, and payload (read-only)
- The proposed attestation's own payload

Predicate semantics:

- **Deterministic**: evaluating the same inputs in two different chain states must produce the same result. The predicate cannot read mutable chain state beyond the explicit input attestations.
- **Bounded compute**: the predicate's gas cost is bounded by `predicate_gas_limit` (default 1000 gas-units; configurable per schema with governance bounds in $[100, 10000]$).
- **Total**: the predicate must terminate on all valid inputs. Non-termination is detected by the gas limit; predicates that exhaust gas are treated as returning `false` and the attestation is rejected at mempool admission.

Examples of useful predicates.

- "Each input attestation's `model_id` field equals my own `model_id` field" (Themisra: prove the attribution chain consistently identifies the same model)

- "Each input attestation's `timestamp` is monotonically older than mine" (audit-trail attestations: ensure causal ordering)
- "The sum of input `amount` fields equals my own `total` field" (financial attestations: conservation-of-value check)

Examples of disallowed predicates.

- "Look up the validator's current reputation and gate on it" (reads mutable state)
- "Walk the attestation graph two hops deep" (unbounded recursion)
- "Hash a 1MB payload via SHA-256" (gas-bounded but potentially out-of-bound for `predicate_gas_limit`)

4.4 Runtime Type Check

When an attestation a of schema σ is submitted, the runtime performs the following checks at mempool admission (before block inclusion):

1. **Schema lookup.** σ exists in Σ at the registered version V_σ. Failure: reject "unknown schema."
2. **Reference resolution.** For each ref $r_i \in \text{refs}_a$, look up the referenced attestation a_i and its schema σ_i, version V_{σ_i}. Failure: reject "dangling reference."
3. **Input-type check.** For each (r_i, a_i, σ_i), verify that $(\sigma_i, v) \in I_\sigma$ for some constraint v admitting V_{σ_i}. Failure: reject "input type mismatch."
4. **Validity check.** For each a_i, verify that a_i.state = VALID. Failure: reject "stale reference" (the input is REVOKED, SLASHED, or DEPENDENT-INVALID).
5. **Predicate evaluation.** Evaluate $P_\sigma(a_1, \ldots, a_n, \text{payload}_a)$. Failure: reject "predicate violated."
6. **Existing PoUA checks.** Threshold signature verification per $\mathcal{A}_\sigma$, fee payment, etc.

All six must pass. Mempool rejection is preferable to block rejection: it avoids consuming block space and gives the submitter a clear failure signal.

Cost analysis. Steps 1-4 are $O(|\text{refs}_a|)$ state lookups; step 5 is bounded by `predicate_gas_limit`; step 6 is the existing PoUA cost. Per-attestation overhead vs leaf attestations is roughly 5-10% at typical reference counts ($|\text{refs}_a| \leq 5$).

Re-checking on cascade. When a referenced attestation transitions out of VALID, dependents already in VALID must be re-evaluated. The cascade rule (§5.4) decides automatic vs lazy re-evaluation. The check itself is identical to the admission-time check, run against the new state.

4.5 Versioning Semantics

When a schema upgrades from $V = k$ to $V = k + 1$, the chain must decide what happens to dependents that referenced $V = k$.

Strict mode. Dependents must re-reference: existing attestations that referenced $V = k$ are unaffected (still valid), but new attestations cannot reference the old version through this schema's input-type set. This is the safest mode: the type contract is enforced at the exact-version level.

Subtyping mode. If $V = k+1$'s payload schema is a structural superset of $V = k$'s (every field in $V = k$ exists with the same type in $V = k+1$), and the predicate P_σ at $V = k+1$ is a "weakening" of P_σ at $V = k$ (every input that satisfied the $V = k$ predicate also satisfies the $V = k+1$ predicate),

then $V = k + 1$ is a **subtype** of $V = k$. Existing dependents that referenced $V = k$ may continue to reference $V = k$ attestations, AND new attestations may reference $V = k + 1$ attestations under the same input-type-set entry.

Default: subtyping for backward-compatible upgrades; strict otherwise. A schema upgrade declares whether it is a subtype of its predecessor. The runtime verifies subtyping claims (structural superset checking is decidable; predicate weakening is detected by re-running P^k against the canonical inputs that satisfied P^{k+1}; if any such input fails P^k, the upgrade is not a subtype).

Per-edge override. A consumer schema's input-type set entry can override the default:

- `(themisra.proof-of-prompt/v1, exact)`: strict, only $V = 1$
- `(themisra.proof-of-prompt/v1, subtype)`: subtyping permitted (default for backward-compatible upgrades)

Migration. When a schema upgrades and existing dependents are not subtype-compatible, the runtime does NOT auto-migrate or auto-invalidate dependents. Existing attestations remain valid. New attestations from those dependents fail the input-type check (§4.4 step 3) until the dependent re-registers with updated input-type-set entries.

---

5. Slashing Propagation

The §3.4 validity state machine introduced DEPENDENT-INVALID as the cascade state. This section specifies the cascade mechanics: what fires when an attestation transitions out of VALID, how dependents respond, how the runtime keeps termination bounded under arbitrary graph shapes, and how concurrent slash events interleave.

5.1 The Cascade Question

When a referenced attestation a transitions to REVOKED, SLASHED, or DEPENDENT-INVALID, the chain has three candidate behaviors for a's dependents:

1. **Strict cascade**: every dependent immediately transitions to DEPENDENT-INVALID at the slash root's transition block.
2. **Lazy cascade**: dependents stay in VALID; the read API exposes the input's invalid status; application logic decides what to do.
3. **Hybrid**: per-schema declaration at registration picks strict or lazy.

None of the three is universally correct. Strict is right when the dependent's claim *relies on* the input's truth and any reader of the dependent should see invalidation propagate (e.g., a financial attestation referencing a sovereign-identity proof). Lazy is right when the dependent's claim is *informationally* dependent but functionally autonomous (e.g., an audit-trail attestation that references a parent event but is itself a standalone receipt of "I observed this"). The hybrid is what we ship: schemas declare their cascade preference at registration (§4.1's `cascade_rule` field), and the runtime enforces it deterministically per-schema.

This section formalizes each candidate, specifies how the cascade algorithm runs, proves the termination theorem (§5.5), and bounds adversarial behavior under concurrent slashes (§5.7).

5.2 Strict Cascade

Under strict cascade, when an attestation a transitions to a non-VALID state at block t, every attestation b such that $a \in \text{refs}_b$ transitions to DEPENDENT-INVALID at block $t + 1$ (or at t if the runtime processes the cascade in the same block; see §5.7 for ordering semantics). The transition is automatic: no on-chain action by the dependent's submitter is required.

Semantics for readers. A read of b after the cascade returns `state = DEPENDENT_INVALID` with metadata `caused_by = a` and `caused_at = t`. Light clients can verify the cascade by re-evaluating: was a a referenced input of b at attestation time, and is a's current state non-VALID? If both yes, b's state is correctly DEPENDENT-INVALID. This is one extra state-tree lookup per cascade depth, $O(1)$ per verification.

Recovery. A dependent submitter cannot un-invalidate b. The recovery path is to submit a new attestation b' with corrected references (pointing to a still-VALID equivalent of a, or omitting the broken reference entirely). b remains DEPENDENT-INVALID forever; b' is a fresh attestation that, if valid, supersedes b for downstream consumers.

Cost. Per slash root, the chain pays for re-evaluating each dependent's state. With strict cascade, this is $O(|\text{descendants}|)$ state-tree writes per slash root, where descendants are the transitive set of attestations reachable through dependency edges from the slash root. §5.5 bounds this in terms of dependency depth.

Use cases. Schemas where the dependent's claim is meaningless if any input is invalid:

- Financial conservation-of-value attestations (the total field depends on all input amounts; if any input is slashed, the total is unverified)
- Identity composition: "this DAO membership references this identity proof"; if the identity is revoked, the membership is no longer evidence-backed
- Multi-party signature: "all parties consented" via references; any party-slash invalidates the multi

5.3 Lazy Cascade

Under lazy cascade, when an input a transitions to a non-VALID state, dependents stay in VALID. The chain records the slash root's transition but does not propagate. The read API returns `state = VALID`, with metadata listing each input's current state. Application code decides what to do with the information.

Semantics for readers. A read of b returns `state = VALID` plus `inputs = [(a, state_a, block_a), ...]`. A reader who wants strict semantics must walk the input states themselves: if any input is non-VALID, treat b as application-layer-invalid. Readers who want lazy semantics ignore the input states and trust b's standalone payload.

Recovery. Not required at the chain level. The dependent submitter may choose to re-submit with corrected references, but the chain does not force it. b remains VALID indefinitely even if every input is slashed.

Cost. Per slash root, the chain pays for the slash root's own transition only. No cascade-induced writes. State-tree footprint is minimal.

Use cases. Schemas where the dependent's claim is independently meaningful and the references are informational:

- Audit-trail attestations: "I observed event X" with X as input; if X is slashed, my observation is still a true record of what I observed
- Bulk AI-provenance attestations: a Themisra session-end attestation that references each turn's individual attestations; if one turn is slashed, the session-end is still a valid record of the session
- Reputation summaries: "this validator did X, Y, Z" where X, Y, Z reference individual events; one event invalidation does not change the summary's accuracy

5.4 Configurable Per-Schema

The schema-declaration object (§4.1) carries a `cascade_rule` field with two values: `STRICT` or `LAZY`. The choice is made by the schema author at registration and applies to all attestations of that schema.

Why per-schema and not per-attestation. Per-attestation cascade rules would require the runtime to read each attestation's rule at cascade time, doubling state-lookup cost. Per-schema rules are read once at schema lookup; subsequent attestations of the schema inherit. The trade-off: less flexibility, more efficiency. A schema author who wants both behaviors registers two schemas (e.g., `themisra.attribution.strict/v1` and `themisra.attribution.lazy/v1`).

Default for new schemas. `STRICT`. This errs on the side of safety: a schema author who has not thought carefully about cascade semantics gets the stronger correctness guarantee. To opt into LAZY, the author must explicitly declare it.

Governance bound. Cascade rule is immutable post-registration; changing it would invalidate the type contract for existing dependents. A schema that wants to switch must register a new schema (with a new schema-id) and let consumers migrate.

Interaction with §4.5 versioning. A schema upgrade may not change cascade rule; the upgrade is rejected if it tries. This is a stricter rule than payload subtyping: cascade rule is a structural property of the schema's economic semantics and cannot be retrofitted.

5.5 Slashing-Cascade Termination Theorem

Claim (Theorem 1). Under any acyclic dependency graph $\mathcal{G}$, when a single attestation a transitions from VALID to non-VALID at block t, the strict-cascade algorithm terminates in $O(d)$ deterministic steps, where d is the maximum depth of any descendant of a in $\mathcal{G}$.

Setup. Define the **descendant set** $D(a) = \{b : a \to^* b \text{ in the dependency graph}\}$ (transitive closure of "references" through strict-cascade edges). The cascade algorithm processes $D(a)$ in BFS order from a:

```
Q := [a]
visited := {a}
while Q non-empty:
    x := Q.dequeue()
    for each y such that x in refs(y) and y.cascade_rule = STRICT:
        if y not in visited:
            transition y to DEPENDENT_INVALID
            visited.add(y)
            Q.enqueue(y)
```

```
return |visited|  # total number of cascaded transitions
```

Proof.

Termination. Each iteration of the while loop removes one element from Q and may add some new elements. An element y is added to Q at most once, because the `if y not in visited` guard prevents re-enqueueing. The total number of enqueue operations is therefore $|D(a)|$, which is finite (the chain has finitely many attestations). The loop terminates.

Bounded steps. The BFS visits $D(a)$ in layer order. Each layer corresponds to a depth-level in $\mathcal{G}$ rooted at a. The maximum number of layers is d by definition. Within each layer, the work per element is $O(\text{outdegree})$ for edge enumeration plus $O(1)$ for the state transition. Summing over layers gives total work $O(|D(a)| \cdot \bar{\text{outdegree}})$. For the BFS depth itself (the number of sequential dependency layers traversed), the bound is exactly d.

Determinism. The BFS order is deterministic given a canonical ordering of attestation ids (by hash). The cascade transitions for the same slash root, processed in the same block, produce identical chain states.

Acyclicity is necessary. Without acyclicity, the BFS could revisit an attestation through a different path. The `visited` set prevents this, but the termination guarantee would still depend on per-edge deduplication; §5.6 addresses the dynamic-cyclic case explicitly.

Corollary (depth bound). At v0, the chain enforces a maximum dependency depth $d_{\max} = 8$ at schema-registration time (cycle-detection in §4.4 already requires walking the graph; depth-bound is computed in the same walk). This bounds any cascade to $\leq |D(a)|$ transitions, with $|D(a)| \leq \text{outdegree}_{\max}^{d_{\max}}$. At typical scale (outdegree ≤ 3, depth ≤ 8), $|D(a)| \leq 6561$ attestations per slash root.

Corollary (cost amortization). The cascade gas cost is charged to the slash root's submitter (the party whose attestation was slashed), not to dependent submitters. This is the §8.3 economic defense against slash-amplification attacks: an adversary who can slash a heavily-referenced attestation faces the cost of cascading all its descendants, even though the descendants are unrelated to the adversary's goal. The cost ratio scales with $|D(a)|$ and the per-attestation transition cost.

5.6 Cycle Handling

§5.5's termination guarantee assumes the dependency graph is acyclic. Two cycle-handling modes are specified.

Static mode (v0 default). Schema registration rejects edges that would close a cycle. Cycle detection is computed at registration time via DFS over the existing graph $\mathcal{G}$ extended with the proposed new edges. The check is $O(|E|)$ time per registration, where $|E|$ is the number of existing edges. At v0 scale ($|E| \leq 1000$), this is microseconds.

A registrant who wants a cyclic dependency must use the dynamic mode (below) or restructure their schemas to break the cycle. The default rejection is the safer choice: cycles are rare in practice and introduce significant complexity in cascade semantics.

Dynamic mode (opt-in, v1+). Cycles are permitted at registration, but each schema must declare additional parameters:

- `cycle_break_rule`: enum `{NO_REVISIT_EDGE, NO_REVISIT_NODE}`. The runtime uses the rule to prevent infinite cascades.
- `max_cascade_depth`: integer in $[1, 100]$. Hard cap on cascade BFS depth, even if the graph allows deeper traversal.

Under `NO_REVISIT_EDGE`, the cascade BFS does not traverse the same edge twice; under `NO_REVISIT_NODE`, it does not visit the same attestation twice. Both prevent infinite loops; `NO_REVISIT_NODE` is stricter and is the recommended default for dynamic-mode schemas.

Why dynamic mode is deferred to v1+. Dynamic cycles complicate the §5.5 termination proof (the depth d becomes the `max_cascade_depth` parameter, not a graph property) and introduce edge cases in concurrent cascades (§5.7). Production use cases for cyclic schemas are rare; v0 ships without dynamic mode and waits for design-partner demand before enabling it.

5.7 Concurrent Invalidation Races

Two slash events can occur in the same block. Consider: attestation a slashed for misbehavior; attestation b revoked by the schema's attestor set; both a and b are referenced by attestation c. What state does c end up in?

Deterministic ordering. Cascade events within a block are processed in canonical attestation-id order (by hash). The runtime sorts the slash roots and processes their cascades sequentially within the block. The order is fully determined by chain state and the block contents; no validator-discretion is introduced.

Deduplication on dependents. When c is reached during the cascade for a, it transitions to DEPENDENT-INVALID with `caused_by = a`. When c is reached again during the cascade for b, the runtime detects c is already DEPENDENT-INVALID (terminal state per §3.4) and skips. The cascade for b continues with c's descendants but does not re-transition c.

Race semantics. The "caused_by" field reflects the first slash root that reached c in canonical order, not necessarily the "logical" root of the invalidation chain. This is a deliberate design choice: chains derive truth from the canonical block ordering, not from external semantics. Applications that need richer attribution can read the full input-state list (the `inputs` metadata from §5.3's lazy-cascade read) to reconstruct the full invalidation pattern.

Gas accounting. Each slash root pays gas for its own cascade traversal. If a and b both reach c, a pays for the transition of c (it gets there first in canonical order); b pays only for its own subtree minus the intersection with a's subtree. This is the §8.3 amplification defense: the second slasher gets a partial discount, but does not get a free ride.

Cross-block races. If a and b are slashed in different blocks, the cascades are sequential by block order. The earlier block's cascade completes (within its block) before the later block's cascade begins. No interleaving between blocks; chain finality enforces the order.

---

6. Use Cases and the Validation Gate

6.1 The Use-Case-Validation Gate

This section is the gate for engineering work on cross-schema composition. The mechanism specified in §3-§5 is, we believe, correct and useful. But correctness is not the same as fit. The engineer-

ing cost of shipping cross-schema composition on Ligate Chain is non-trivial: §4.4 admission-time checks add latency to every attestation submission; §5 cascade machinery adds state-tree complexity; §7.4 light-client verification requires a graph-walk per dependent. The cost is paid by every workload on the chain, not just the workloads that compose.

We will not pay this cost on speculation. The engineering cycle for cross-schema composition begins only when at least 2-3 design partners have submitted concrete use-case descriptions matching the §6.2 template, with each description identifying:

- a real workload (existing or planned) that requires cross-schema references
- a concrete failure mode if the references remain application-level (not chain-enforced)
- a willingness to integrate against the v2 protocol during the early-stage pilot

Until then, this paper documents the design space, the security analysis, and the comparison with prior art. The mechanism is specification-only.

This is a deliberate process choice. "Schemas as composable Lego" is a vibe; concrete workflow X needing schemas A, B, C to compose under chain-enforced typing is a use case. The paper exists so that, when the second category materializes, the design is captured and ready. Until it does, the chain ships with single-schema attestations only and the four flagship products at v1 (Themisra, Mneme, Iris, Atlas) operate without this primitive.

6.2 Use Case Template

For a design partner to validate the use case, they should submit a description in this form:

Field 1: Consumer workflow. A one-paragraph description of the application or product layer that needs to produce attestations referencing other attestations. Avoid abstractions; describe the actual product behavior.

Field 2: Input schemas required. Which existing or planned Ligate Chain schemas does the consumer reference? Be specific: schema name, version, and which fields of the input payload the consumer's predicate depends on.

Field 3: Why chain-enforced typing is required. A specific failure mode that arises if the typing remains application-level. Two acceptable answers: (a) the failure mode is a security risk (type confusion, slash bypass, etc.) that the chain-level check would prevent, or (b) the failure mode is a cross-app correctness issue (one consumer's check disagrees with another's) that chain-level consistency would prevent.

Field 4: Slashing-cascade preference. Strict, lazy, or "we need both for different consumer schemas." With reasoning. If strict: what is the value of the dependent attestation if the input is invalid? If lazy: how does the application handle the partial-validity case at the read API?

Field 5: Failure mode if the chain doesn't enforce. What does the partner do today, in the absence of this mechanism? Either (a) work around it at the application layer (with effort estimate) or (b) avoid the use case entirely (with cost estimate of the lost use case).

Field 6: Integration commitment. Does the partner agree to integrate against the v2 protocol during the early-stage pilot? With which timeline?

A use-case description that does not address fields 3 and 5 is not enough. The gate is about *whether the mechanism is justified*, not just whether someone could imagine using it. Fields 3 and 5 force

the partner to articulate the case for chain-enforcement vs application-enforcement, which is the harder question.

6.3 Hypothetical Use Cases (Not Yet Validated)

The following three hypotheticals are *not* validated use cases. They illustrate the kinds of workflows the mechanism is designed for. They are not commitments to ship until at least 2-3 design partners submit §6.2 descriptions for them or similar.

Hypothetical 1: AI provenance attribution.

Consumer schema: a Themisra `proof-of-attribution` schema at v1 produces "AI output O was generated by model M from prompt P."

Inputs: a Themisra `proof-of-prompt` schema at v1 (the prompt attestation) and a Themisra `model-registration` schema at v1 (the model registration).

Why chain-enforced: §1.3 worked this through. A type-confusion attack on attribution attestations would let an adversary substitute arbitrary chain state for a prompt attestation, creating attribution claims that are structurally bogus but parseable by downstream readers. Type confusion at the attribution layer is a safety issue for any application that consumes attribution attestations to assign responsibility.

Cascade preference: Strict. If a prompt attestation is slashed (e.g., the prompt's claimed timestamp is forged), all attribution attestations referencing it are no longer evidence of provenance and should be flagged immediately.

Validation status: hypothetical. Themisra's product roadmap includes attribution attestations but does not currently require cross-schema composition; the v1 product ships with single-schema proof-of-prompt only.

Hypothetical 2: Multi-party consent.

Consumer schema: `multi-party-consent/v1` produces "parties P1, P2, P3 all consented to action A."

Inputs: k instances of `individual-consent/v1`, one per party.

Why chain-enforced: a multi-party-consent attestation is only meaningful if each input is a valid individual-consent attestation. Application-level checks could miss one of the inputs, producing a multi-party-consent attestation that claims unanimous consent when only some parties consented. Type-confusion would be a privilege escalation in any workflow that gates actions on unanimous consent.

Cascade preference: Strict. If any individual consent is revoked (e.g., a party retracts), the multi-party-consent attestation is no longer valid evidence of unanimity.

Validation status: hypothetical. No design partner has asked for this specific composition. Multi-party consent flows on Ligate Chain at v1 use single-schema attestations with an off-chain aggregation step.

Hypothetical 3: Proof of audit.

Consumer schema: `proof-of-audit/v1` produces "this RWA reserve was audited by this licensed auditor on this date."

Inputs: an `auditor-license` schema at v1 (the auditor's qualified-attestor credential) and a `reserve-snapshot` schema at v1 (the audited document attestation).

Why chain-enforced: an audit attestation is only meaningful if (a) the auditor's license is valid at audit-time and (b) the reserve snapshot is a legitimate attestation, not a forgery. Type confusion on either input lets an adversary produce bogus audit claims that downstream applications (regulators, counterparties) might rely on for compliance decisions.

Cascade preference: Strict. If the auditor is later slashed for misbehavior on any prior audit, pending audit attestations referencing the auditor's license should transition to DEPENDENT-INVALID. Regulators reading the chain see the invalidation and can re-request audit from a different licensed auditor.

Validation status: hypothetical. Discussions with compliance-focused design partners are at an early stage; no concrete §6.2 description has been submitted.

6.4 What Section 6 Looks Like at v0.2.x

When 2-3 design partners submit §6.2 descriptions, §6.3 expands to include the validated use cases with explicit partner attribution, integration timelines, and any deviations from the v0.2 mechanism that the partners require. At that point the §6.1 gate is satisfied and the engineering cycle for v2 cross-schema composition begins.

Until then, §6.3 hypotheticals are illustrative, not prescriptive. The paper is design-space documentation; the chain ships with single-schema attestations.

---

7. Comparison with Prior Systems

Cross-schema composition with chain-enforced typing occupies a distinct point in the design space against the comparators surveyed in §2. This section compares them on the axes that matter for application correctness, security, and engineering cost.

Axis	Ethereum smart-contract refs	EAS schema graph	Capability-secure languages	Dependent types (Coq/Agda)	Cross-schema composition (this paper)
Where typing is enforced	Application (consumer contract)	Application (consumer logic)	Compiler (compile time)	Compiler (proof-search)	Chain runtime (admission)
Type expressiveness	High (EVM logic)	Medium (structural)	High (full PL types)	Maximum (dependent)	Medium-high (structural + predicate)
Type-check cost	Per-call gas	Application-level	Compile-time (once)	Compile-time (undecidable)	Mempool check, $O(\|\text{refs}\|)$
Slash propagation	Application-level	Application-level	N/A	N/A (compile-time)	Chain runtime, BFS cascade $O(d)$ (§5.5)
Cycle handling	Application choice	Application choice	Acyclicity via authority graph	Acyclicity via well-founded recursion	Static rejection by default (§5.6); dynamic mode v1+
Light-client verifiability	Hard (per-contract logic)	Hard (must replay app logic)	N/A (off-chain)	N/A (off-chain)	Easy ($O(1)$ per cascade level)
Production status	Live (Ethereum mainnet)	Live (Ethereum mainnet, 2022+)	Live (research languages + Caja)	Live (research community)	v2 specification (this paper)

The four comparators each solve a different subset of the problem. **Ethereum smart-contract references** and **EAS** push typing to the application layer; the cost is repeated implementation and audit per consumer. **Capability-secure languages** push typing to the compiler; the cost is they live off-chain and have no model for revocation or runtime invalidation. **Dependent types** push typing to maximum expressiveness; the cost is they require proof-search at type-check time, which is impractical for a chain runtime.

Cross-schema composition occupies a different design point: typing is enforced at the chain runtime, at mempool admission, with bounded compute per attestation. The trade-off is that type expressiveness is constrained: schemas declare structural payload schemas plus a bounded-compute predicate, not arbitrary contract logic. Section 1.3 argued this constraint is the right one for an attestation-native chain: chain-level typing is provable (§5.5 theorem), the type contract is part of consensus, and the §8 security analysis is tractable.

The unique value-add of this paper's design is the **slashing-cascade machinery**. None of the four comparators model revocation propagation through references. EAS has revocation flags but no cascade. Capability languages have no revocation concept. Dependent types operate at compile time and have no runtime invalidation. This paper specifies the cascade explicitly (§5), proves termination (§5.5), and handles concurrent invalidation races (§5.7) with deterministic ordering. The cascade machinery is what makes cross-schema composition safe in a production chain.

---

7. Comparison

7.1 vs Ethereum Smart-Contract References

[**v0.2:** EVM contracts reference state by hash; type-checking is application-level. Quantitative: gas cost per reference, type-soundness, slash propagation cost.]

7.2 vs EAS Schema Graph

[**v0.2:** EAS supports schema-references via UID. Type-checking is application-level (no chain enforcement). Slash propagation is application-level (no chain cascade).]

7.3 vs Capability-Secure Languages

[**v0.2:** E / Pony / Joe-E provide compile-time capability checks. Closer in spirit to this paper's typing but operates at language level, not chain level. Conceptual ancestor; not directly comparable.]

7.4 vs Cosmos IBC Light Clients

[**v0.2:** IBC supports cross-chain references with light-client verification. Different problem (cross-chain) but informs the on-chain encoding of "this attestation is provably from chain X at height H."]

---

8. Security Analysis

We analyze cross-schema composition against three attack families. Each family bounds an adversary's behavior under the mechanism's runtime checks and economic guarantees. The §5.1 cost-to-grind argument from PoUA carries over; this section focuses on the attacks that are unique to cross-schema composition.

8.1 Threat Models

Three attack families exhaust the surface that cross-schema composition introduces beyond single-schema attestations:

1. **Type-confusion attacks**: adversary submits an attestation that references an input of the wrong type, hoping a consumer or downstream reader will accept the malformed claim.
2. **Slash-amplification attacks**: adversary triggers a slash on a heavily-referenced attestation, hoping the cascade disrupts unrelated workflows that happened to reference the slashed attestation.
3. **Cycle-induced DoS**: adversary registers a cyclic dependency graph (under dynamic mode) or constructs cascade interleaving (under concurrent invalidation) hoping the cascade stalls, loops, or exhausts resources.

For each family, we specify the defense, bound the attack cost, and identify the residual risk.

8.2 Type-Confusion Defense

Attack. An adversary submits attestation a of schema σ with $\text{refs}_a = (\text{att-id})$, where `att-id` points to an attestation of a different schema σ' that is not in I_σ. The adversary hopes the consumer's predicate P_σ will parse the wrong-type input "well enough" to return `true`, accepting the bogus attestation. Or, alternatively, that downstream readers will trust the chain's acceptance and use the bogus attestation as canonical.

Defense (structural, in-protocol). §4.4 step 3 (input-type check) rejects a at mempool admission. The runtime resolves `att-id` to its schema σ' and tests $(\sigma', V_{\sigma'}) \in I_\sigma$. If not, admission is denied; a never lands. The attacker pays the mempool-admission cost (small, bounded by the chain's spam-protection mechanism) and gets nothing.

Bound. Type-confusion is fully prevented at admission. There is no economic exposure beyond the admission-attempt cost. The defense is structural: no parameter calibration, no adversary-cost analysis required.

Residual risk: schema author error. If the schema author registers I_σ with an entry that admits the "wrong" schema (e.g., declares `*` wildcard version constraint, or includes a schema with overlapping payload structure), the defense weakens. The chain accepts inputs the author intended to exclude. This is the schema author's responsibility; the chain offers the `exact` and semver-range constraints to make tight typing easy.

Residual risk: predicate weakness. Even with tight `input_type_set`, a poorly-written predicate P_σ might return `true` for inputs that semantically should not validate. This is application-correctness, not protocol-correctness. The chain enforces type contracts but not predicate-content correctness.

8.3 Slash-Amplification Defense

Attack. An adversary identifies a heavily-referenced attestation a (one with many descendants $|D(a)| \gg 1$ under the §5.5 BFS). The adversary triggers a slash on a via misbehavior they orchestrate (e.g., they control or compromise a's attestor set, or they detect a slashable violation a has committed). The cascade fires through $D(a)$, transitioning every descendant to DEPENDENT-INVALID. The adversary's goal is not the slash itself but the disruption: legitimate users of dependent schemas find their attestations invalidated through no fault of their own.

Defense (economic). Per §5.7's gas accounting, the slash root pays the cascade cost. An adversary who triggers a slash on a pays gas proportional to $|D(a)|$ for the cascade. Heavily-referenced attestations are expensive to slash. The economic bound:

$$\text{cost}_{\text{slash}}(a) = c_{\text{base-slash}} + c_{\text{per-cascade-step}} \cdot |D(a)|$$

where $c_{\text{per-cascade-step}}$ is the chain's per-cascade-transition gas cost. For an adversary's attack to be net-positive, the value the adversary extracts (from disrupting dependent workflows) must exceed the cascade gas cost they pay. At v0 calibration ($c_{\text{per-cascade-step}} \sim 10^3$ gas-units, $|D(a)| \leq 6561$ at $d_{\max} = 8$, max outdegree 3), the worst-case slash payment is on the order of 7×10^6 gas-units, comparable to a moderate contract deployment. Disrupting unrelated workflows that don't pay the adversary back is bad business.

Defense (structural). $d_{\max} = 8$ at v0 (§5.5 corollary) caps cascade depth regardless of graph shape. Even an adversary who carefully crafts a deep-dependency-chain attack is limited.

Bound. Per-slash adversary cost is linear in $|D(a)|$, which is bounded by depth and outdegree. The mechanism makes deep-graph slash attacks expensive enough to deter most adversaries; the defense is economic, not structural.

Residual risk: free-rider slashes. If the adversary detects a slashable violation on a that someone else would have reported anyway (or that the chain would have caught automatically), they get the cascade-disruption "free." The chain's slashing protocol rewards the reporter (per PoUA §4.5) but does not require the reporter to pay the cascade cost. v0.2 of this paper accepts this asymmetry: it's a feature, not a bug, that legitimate slash-reporting is incentivized even when the cascade fires.

8.4 Cycle-DoS Defense

Attack. Two sub-attacks under this family.

Sub-attack A: static-cycle attempted at registration. Adversary registers schemas A, B, C with $A \to B \to C \to A$ dependency cycle. If the chain accepts the registration, cascades through any of A, B, C could loop indefinitely (each invalidation triggers the next, which triggers the next, which retriggers the first).

Sub-attack B: dynamic-mode cascade overload. Under dynamic mode (§5.6), an adversary registers schemas with `cycle_break_rule = NO_REVISIT_NODE` but constructs a cascade chain that interleaves cycles in a way that maximizes cascade computation (e.g., diamond patterns where the BFS still visits many edges before deduplication).

Defense (structural, sub-attack A). §5.6's static cycle detection at registration runs DFS over $\mathcal{G}$ extended with the proposed edges. Any cycle is detected before the registration commits; the registration is rejected. Cost: $O(|E|)$ per registration, microseconds at v0 scale.

Defense (structural + economic, sub-attack B). Under dynamic mode, `max_cascade_depth` caps the BFS regardless of graph shape. `cycle_break_rule` prevents revisits. Worst-case cascade work is $O(\text{outdegree}_{\text{max}}^{\texttt{max\_cascade\_depth}})$ which is bounded by the schema author's declared depth limit. The chain charges gas per cascade step (§8.3), so even a maximal-cost cascade costs the slash root real money.

Bound. Static mode: cycle attacks are fully prevented at registration. Dynamic mode (v1+): cascade attacks are bounded by `max_cascade_depth` and economically deterred by per-step gas.

Residual risk: design choice in dynamic mode. A schema author who picks an aggressive `max_cascade_depth` (e.g., 100, the protocol-bound maximum) makes cascades expensive for the entire chain when they fire. Other dependents of the same root could pay the inflated cost. v0.2 keeps dynamic mode out of v0; v1+ specifies governance bounds for `max_cascade_depth` based on operational experience.

8.5 Composition with Other Threat Models

Three observations on how cross-schema composition interacts with the threat models from companion papers.

With native-delegation: a hot-key compromise on a schema with cross-schema dependencies inherits the §8.2-§8.4 bounds. The native-delegation paper §8.5 scope-predicate defense already bounds the hot key's authority to enumerated schemas; combined with this paper's typing, the cross-schema attack surface from a compromised hot key is the intersection of the grant scope and the schema's declared input-type set. Both layers must agree; the more restrictive bound wins.

With per-schema fees: §5.5's cascade cost is charged to the slash root at the slash root's schema's base fee. A schema with high b_σ (heavy traffic, congested) is expensive to slash and therefore expensive to cascade-attack. This creates an emergent property: high-volume schemas are more cascade-resistant than low-volume ones, because attacker cost scales with the slash root's b_σ. We do not consider this a security feature (it should not be relied upon), but it is a side effect worth flagging.

With PoUA reputation: cascade-disrupted dependents' submitters do not have their reputation slashed for the disruption. The §4.3 reputation update is per-attestation at admission time; an attestation that subsequently becomes DEPENDENT-INVALID via cascade does not retroactively penalize the submitter. Reputation is for behavior at submission time; cascade is for state after submission. The two are independent surfaces.

---

9. Limitations and Future Work

The v0.2 mechanism specifies single-chain, structural-typing-plus-bounded-predicate, BFS-cascade composition. Four extensions remain out of scope.

9.1 Cross-Chain Composition

References across IBC-connected chains require light-client proofs of the input attestation's state on the source chain. The mechanics: IBC packet carrying the input attestation, its Merkle proof against the source chain header, and a freshness commitment. The complications: IBC update latency makes cross-chain state stale by the round-trip; revocation on the source chain is not immediately visible to dependents on the counterparty chain; cascade semantics need re-validation when state crosses chain boundaries. Each is a separable problem; together they constitute a follow-up paper.

For v0.2, the recommendation is to compose at the application layer: an off-chain coordinator can stitch attestations from multiple chains, paying the trust cost of the coordinator. The unified primitive is future work.

9.2 Predicate Expressiveness

Type predicates P_σ are deterministic, bounded-compute boolean functions. More expressive predicates would unlock new use cases:

- **Recursive predicates.** A predicate that walks the dependency graph two hops deep ("the input's input has property X"). v0.2 disallows; future work could add bounded recursion with explicit depth limits.
- **Zero-knowledge predicates.** A predicate that verifies a SNARK or STARK proof about the input. v0.2 disallows for compute-cost reasons (proof verification at admission time is too expensive); future work with proof-aggregation could reduce per-attestation cost.
- **Probabilistic predicates.** A predicate that samples from a randomness source. v0.2 disallows because determinism is a hard requirement; future work could specify deterministic-randomness sources (chain-block-hash beacons) and bounded sampling.

9.3 Proof-Carrying Attestations

Rather than reference inputs by attestation-id and re-evaluate the predicate, an attestation could carry a SNARK proof that "the inputs satisfied the predicate." Light clients verify the SNARK; the chain stores the proof. This trades on-chain state (the ref-list) for proof size and verification cost. Different design point with different tradeoffs; deferred to future work.

9.4 Privacy-Preserving References

Public chain state reveals the dependency graph: anyone can read which schemas reference which. For use cases requiring private references (e.g., a competitive-intelligence audit attestation that does not want to disclose which data sources it consulted), the mechanism needs a privacy layer. Two paths: commit-reveal (the consumer commits to the input-id before knowing the input is correct, then reveals later), or zero-knowledge composition (the consumer proves it has *some* valid input without disclosing which one). Both are research-grade; not a v0.2 commitment.

9.5 Schema-Author Reputation

A schema with high ρ_σ (per the per-schema fees paper) extracts more fee revenue from dependents. A malicious schema author could register a popular schema, attract dependents, then start producing low-quality attestations to capture fees. This is a schema-author reputation problem orthogonal to cross-schema composition: PoUA tracks per-validator reputation, not per-schema-author.

A follow-up paper could specify schema-author reputation that tracks attestation correctness over time; v0.2 of cross-schema composition does not include it.

---

10. Conclusion

Cross-schema composition with chain-enforced typing and slashing-aware proof propagation is the right primitive for an attestation-native chain that wants to scale beyond single-schema workloads. The §1.1 composition thesis is the central motivation: attestations are claims about things, and at production scale, many natural claims are claims about other claims. The §1.3 type-confusion problem is the central justification: without chain-level typing, every consumer re-implements the type check, errors compound, and audit costs scale linearly in composition depth.

The paper's four contributions resolve the design space. (1) **Mechanism (§3 + §4)**: schemas as typed-graph nodes with declared input-type sets, semver-constrained edges, bounded-compute type predicates, and admission-time runtime type checks. (2) **Slashing-cascade termination theorem (§5.5)**: under acyclic graphs, the BFS cascade terminates in $O(d)$ deterministic steps with per-dependent deduplication and gas charged to the slash root. (3) **Security analysis (§8)**: three attack families (type confusion, slash amplification, cycle DoS) bounded by structural and economic defenses, with composition properties documented against native delegation, per-schema fees, and PoUA reputation. (4) **Use-case validation gate (§6)**: explicit framing that engineering work begins only when 2-3 design partners submit concrete use-case descriptions matching the §6.2 template.

The mechanism is positioned as a **v2 protocol feature**, not v1 day-1. Ligate Chain v0 ships with single-schema attestations; the four flagship products at v1 (Themisra, Mneme, Iris, Atlas) operate on single-schema attestations only. Cross-schema composition lands when the §6.1 gate is satisfied. This paper documents the design space and the security argument so that, when the gate opens, engineering work has a target to ship against.

What this paper does not do. It does not advocate for shipping cross-schema composition on day one. It does not claim that workflows currently using single-schema attestations should switch. It does not commit Ligate Chain to a v2 release date. It documents what the mechanism would look like, why the mechanism would be correct, and what the security argument would be, in the conditional voice the v2 status warrants.

What this paper does do. Capture the design space at the point in time when the trade-offs are fresh, the comparison with prior art is current, and the integration with companion primitives (native delegation, per-schema fees, PoUA) is concrete. The mechanism is small, the theorem is tight, and the composition with adjacent primitives is orthogonal. If and when design-partner demand validates the use cases, the engineering cycle has a reference document to start from.

Invitations. The paper is open to external review. The §6.2 use-case template is open to design-partner submissions: cold-asks for use-case descriptions are open through `hello@ligate.io`. Feedback on the §5.5 theorem (especially edge cases around cycle handling in dynamic mode) is welcome from researchers in distributed systems and capability-secure systems.

The §1.4 central question was: what is the minimum typing and slashing-propagation primitive that makes cross-schema composition safe to use in production, while remaining cheap enough to ship in a runtime, and gated cleanly enough that workloads which do not need it pay no overhead? This

paper answers: structural typing with bounded-compute predicates, BFS cascade with deterministic ordering, static-by-default cycle handling, and an explicit use-case validation gate. The first three are mechanism; the fourth is discipline. Both matter.

References

Account-abstraction and smart-contract reference patterns.

- Ethereum community (2018+). *ERC-721: Non-Fungible Token Standard*. https://eips.ethereum.org/EIPS/eip-721
- Ethereum community (2018+). *ERC-1155: Multi Token Standard*. https://eips.ethereum.org/EIPS/eip-1155
- Ethereum community (2022+). *ERC-4907: Rental NFT Standard*. https://eips.ethereum.org/EIPS/eip-4907

Attestation systems.

- Ethereum Attestation Service (EAS). Documentation and contracts. https://attest.org/
- Ceramic Network. https://ceramic.network/

Capability-secure programming languages.

- Miller, M. (2006). *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. PhD thesis, Johns Hopkins University. (E language)
- Pony language. https://www.ponylang.io/
- Mettler, A., Wagner, D. (2008). *The Joe-E Language Specification (Version 1.0)*. UC Berkeley Tech Report. Available from the Berkeley Computer Security Group archive.

Dependent types.

- Bertot, Y., Castéran, P. (2004). *Interactive Theorem Proving and Program Development: Coq'Art: The Calculus of Inductive Constructions*. Springer.
- Norell, U. (2007). *Towards a practical programming language based on dependent type theory*. PhD thesis, Chalmers. (Agda)
- Brady, E. (2017). *Type-Driven Development with Idris*. Manning Publications.
- de Moura, L. et al. (2015). *The Lean Theorem Prover (system description)*. CADE 2015.

Distributed-systems invalidation patterns.

- Mohan, C., et al. (1992). *ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging*. ACM TODS.
- SQL Standard: `ON DELETE CASCADE` semantics. ISO/IEC 9075.

Companion Ligate Labs research.

- Ligate Labs (2026). *Proof of Useful Attestation: Consensus-Weighting Primitive for Attestation-Native Chains*. Working paper v0.8.
- Ligate Labs (2026). *Native Delegation as a Runtime Primitive*. Working paper v0.2.
- Ligate Labs (2026). *Per-Schema Fee Markets for Attestation-Native Chains*. Working paper v0.2.
- Ligate Labs (2026). *Schema-Bound Tokens: AttestorSet as Mint Authority*. Working paper v0.1.

Chain stack.

- Sovereign Labs (2024). *Sovereign SDK*. https://github.com/Sovereign-Labs/sovereign-sdk
- Celestia Labs (2023). *Celestia: Modular Data Availability*. https://celestia.org/learn/
- Inter-Blockchain Communication (IBC) protocol specification. https://github.com/cosmos/ibc

---

Appendix A: Simulator Validation Plan

A reference simulator under `prototypes/cross-schema-composition-sim/` (planned milestone M1, after this paper lands and the §6 gate is satisfied) will provide cross-language test vectors for the canonical mechanisms in this paper. The simulator follows the v0.7-PoUA + v0.2 native-delegation + v0.2 per-schema-fees discipline: every numerical claim and every theorem in this paper gets a corresponding simulator test.

Planned modules under `src/cross_schema_composition_sim/`:

- `schema.py`: the §3.1 typed-schema model (`Schema`, `InputTypeEntry`, `SchemaDeclaration`).
- `graph.py`: the §3.2 dependency graph with cycle detection (DFS for static mode, plus dynamic-mode utilities).
- `attestation.py`: the §3.3 attestation tuple with the §3.4 validity state machine.
- `typecheck.py`: the §4.4 runtime type check (schema lookup, reference resolution, input-type check, validity check, predicate evaluation).
- `cascade.py`: the §5 BFS cascade with per-dependent deduplication and the §5.7 deterministic ordering.
- `security.py`: the §8 attack-cost analysis (type-confusion attempt cost, slash-amplification cost as a function of $|D(a)|$, cycle-DoS bound under static/dynamic mode).

Planned test coverage:

- §3.4 validity state machine transitions
- §4.4 admission-time type check correctness (positive + negative cases)
- §5.5 termination theorem at canonical graph shapes (linear, tree, diamond)
- §5.6 cycle detection at registration (static mode), `max_cascade_depth` enforcement (dynamic mode)
- §5.7 concurrent-invalidation race deduplication and gas accounting
- §8.3 slash-amplification cost as a function of descendant set size

Cross-language test vectors as JSON files under the simulator's `test_vectors/` directory, matching the format used by `per-schema-fees-sim`: each vector has `input`, `expected`, and `tolerance` fields. Any future Rust or TypeScript implementation can verify identical outputs.

The simulator is **not** part of v0.2 of this paper. It is named in this appendix so the §6 gate's engineering scope is clear: when the use-case-validation gate is satisfied, the M1 simulator milestone is the first deliverable, before the chain implementation work.

Appendix B: Formal Definitions

We collect the formal definitions used throughout the paper in one place.

Definition (Schema). A tuple $\sigma = (I_\sigma, O_\sigma, P_\sigma, \mathcal{A}_\sigma, V_\sigma)$ where I_σ is the input-type set (finite list of (schema-id, version-constraint) pairs), O_σ is the structural payload schema, P_σ is the deterministic type predicate, $\mathcal{A}_\sigma$ is the attestor set, and V_σ is the schema version.

Definition (Dependency graph). $\mathcal{G} = (\Sigma, E)$ where Σ is the set of registered schemas and $E \subseteq \Sigma \times \Sigma$ is the set of dependency edges with $\sigma_a \to \sigma_b \in E$ iff $(\sigma_b, v) \in I_{\sigma_a}$ for some v that admits V_{σ_b}.

Definition (Attestation). A tuple $a = (\sigma, K^{\text{signer}}, \text{payload}_a, \text{refs}_a, t_a, s_a)$ where σ is the schema-id, K^{signer} is the signing key, payload_a conforms to O_σ, refs_a is the reference list, t_a is the inclusion height, and s_a is the threshold signature.

Definition (Validity state). For each attestation a, state(a) is one of: VALID, REVOKED, SLASHED, or DEPENDENT-INVALID. State transitions follow §3.4. Terminal states (no further transitions): REVOKED, SLASHED, DEPENDENT-INVALID.

Definition (Descendant set). For an attestation a and a dependency graph $\mathcal{G}$, $D(a) = \{b : a \to^* b\}$ is the set of attestations reachable through `refs` edges from a.

Definition (Cascade BFS). The §5.5 algorithm: enqueue a; while queue non-empty, dequeue x, for each y with $x \in \text{refs}(y) \wedge \text{cascade-rule}(y) = \text{STRICT} \wedge y \notin$ visited: transition y to DEPENDENT-INVALID, add y to `visited`, enqueue y. Termination guaranteed under acyclic $\mathcal{G}$ in $|D(a)|$ enqueue operations bounded by $\text{outdegree}_{\max}^{d_{\max}}$.

Definition (Cycle-break rule). For dynamic-mode schemas (§5.6), one of `NO_REVISIT_EDGE` or `NO_REVISIT_NODE`. The runtime tracks edge or node visits during the cascade and skips revisits per the rule.

Definition (Slashing-cascade termination theorem). Under acyclic dependency graph $\mathcal{G}$, the strict-cascade algorithm initiated at attestation a terminates in $O(d)$ deterministic steps where d is the maximum depth of any descendant of a in $\mathcal{G}$. Proof: §5.5.

---

9. Limitations and Future Work

9.1 Cross-Chain Composition

[**v0.2:** Out of scope. A separate paper covers attestation-graph references across IBC-connected chains.]

9.2 Predicate Expressiveness

[**v0.2:** Type predicates P_σ are bounded-compute boolean functions. More expressive predicates (zero-knowledge, recursive) are research-grade and deferred.]

9.3 Proof-Carrying Attestations

[**v0.2:** Attestations could carry SNARK proofs of input validity rather than referencing. Different design point with different tradeoffs; not a v0.2 deliverable.]

9.4 Privacy-Preserving References

[**v0.2:** A reference reveals the dependency edge in the public chain state. Use cases requiring private references (e.g., consumer schema does not want to disclose which input it consumed) need additional cryptography. Future work.]

---

10. Conclusion

[**v0.2:** Recap. Chain-enforced typing for attestation references, slashing-aware proof propagation through the schema dependency graph, and explicit non-shipment in v1. The mechanism is interesting and probably correct, but the engineering cost is justified only by validated design-partner demand. v2 territory.]

References

[**v0.2:** Ethereum Attestation Service docs, capability-secure languages survey, dependent types references (Coq / Agda / Idris / Lean), cascading-delete semantics in DB literature, plus standard PoUA references.]

Appendix A: Simulator Validation Plan

[**v0.2:** What `prototypes/cross-schema-composition-sim/` will contain. Test harness for type-check correctness, slashing-cascade termination, cycle-detection, concurrent-invalidation race deduplication. Cross-language test vectors for the canonical type-check predicate encoding.]

Appendix B: Formal Definitions

[**v0.2:** Restated definitions of schema, dependency edge, attestation, type predicate, validity state, cascade rule, in formal notation.]